

# AADK 12: Task 3 — Theme + Animation

## Integration: Building a Polished UI Module

**Name:** Rajat Prakash Dhal

**Course/Unit:** Android Development with Kotlin – Session 12

**USN:** 1hk22cs115

**email:** [1hk22cs115@hkbk.edu.in](mailto:1hk22cs115@hkbk.edu.in)

### 1. Define the Module

**Module Name:** Animal Gallery with Expand/Collapse and Favourite

This module displays a **gallery of animal cards** where users can browse animals in a scrollable list. Each card shows an image and name, and users can tap the card to expand it for additional details such as habitat and diet. Users can also mark animals as favourites using a heart icon.

This module is visually important because it combines **theme-based design (Material 3 colours, shapes, typography)** with **interactive animations** that make the UI feel smooth, responsive, and modern.

---

### 2. Data Model & State Variables

#### Data Model

```
data class Animal(  
    val id: Int,  
    val name: String,  
    val imageUrl: String,  
    val habitat: String,  
    val diet: String,  
    var isFavourite: Boolean = false  
)
```

This model represents each animal card displayed in the gallery.

---

## State Variables

| State Variable    | Type    | Purpose                     |
|-------------------|---------|-----------------------------|
| isExpanded        | Boolean | Controls card expansion     |
| isFavourite       | Boolean | Tracks favourite status     |
| currentThemeMode  | Boolean | Determines light/dark theme |
| animationProgress | Float   | Tracks animation progress   |

Example state setup:

```
var isExpanded by remember { mutableStateOf(false) }
```

```
var isFavourite by remember { mutableStateOf(false) }
```

---

## Theme Token Mapping

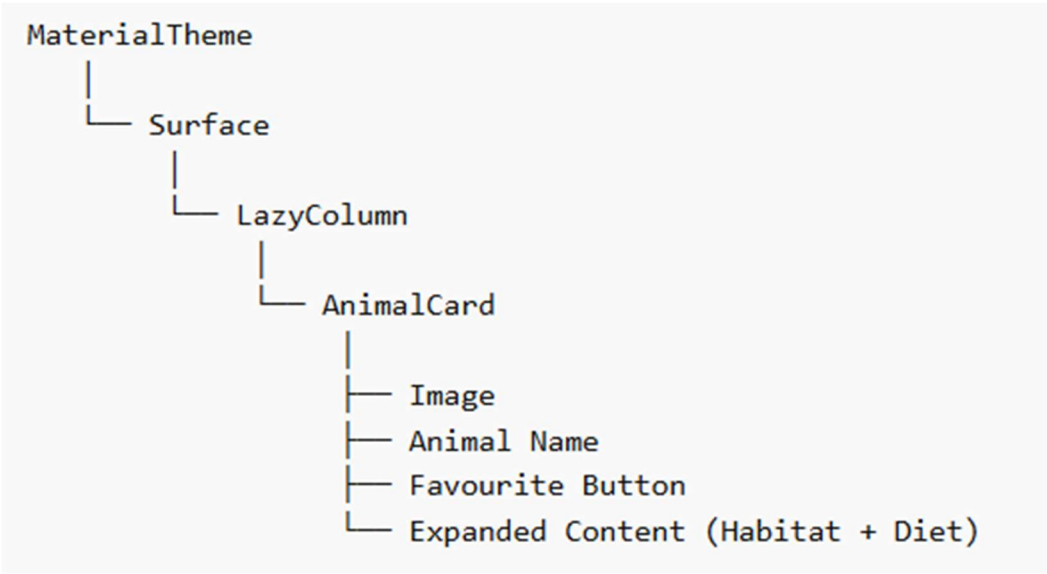
| UI Element               | Theme Token                |
|--------------------------|----------------------------|
| Favourite icon colour    | colorScheme.secondary      |
| Expanded card background | colorScheme.surfaceVariant |
| Card background          | colorScheme.surface        |
| Title text               | typography.titleMedium     |
| Card shape               | shapes.medium              |

This ensures UI consistency and avoids hard-coded styles.

---

### 3. UI Composition Plan

#### Layout Structure



#### Key Composables Used

| Composable    | Purpose              |
|---------------|----------------------|
| MaterialTheme | Applies global theme |
| Surface       | Background container |
| LazyColumn    | Scrollable list      |
| Card          | Animal item display  |
| Image         | Display animal photo |
| IconButton    | Favourite toggle     |

#### Example UI Snippet

```
Card(
    modifier = Modifier
        .fillMaxWidth()
        .padding(8.dp)
```

```

        .animateContentSize(),
        shape = MaterialTheme.shapes.medium,
        colors = CardDefaults.cardColors(
            containerColor = MaterialTheme.colorScheme.surface
        )
    ) {
        Column {
            Text(
                text = animal.name,
                style = MaterialTheme.typography.titleMedium
            )
        }
    }
}

```

---

### Animation Triggers

| Trigger       | Animation           | Effect                   |
|---------------|---------------------|--------------------------|
| Tap card      | animateContentSize  | Card expands smoothly    |
| Tap favourite | animateColorAsState | Heart icon colour change |
| Press card    | animateFloatAsState | Slight scale feedback    |

Example:

```

val iconColor by animateColorAsState(
    targetValue = if (isFavourite)
        MaterialTheme.colorScheme.secondary
    else
        Color.Gray
)

```

---

## 4. Implementation Timeline & Dependencies

### Development Steps

#### Step 1 — Theme Setup

- Define colour palette, typography, shapes
- Apply `MaterialTheme` wrapper

#### Step 2 — Static Layout

- Build `LazyColumn` list
- Create basic `AnimalCard` composable

#### Step 3 — Add Animations

- Implement card expansion
- Add favourite toggle animation
- Add scale feedback on click

#### Step 4 — Testing & Optimization

- Test light/dark theme
- Test animation responsiveness
- Verify performance in long lists

---

### Dependencies

| Library           | Purpose                 |
|-------------------|-------------------------|
| Coil              | Load images efficiently |
| Material3         | UI components           |
| Compose Animation | Built-in animation APIs |

Example dependency:

```
implementation("io.coil-kt:coil-compose:2.4.0")
```

---

## 5. Quality, Accessibility & Performance Checklist

### Quality Checks

- ✓ Use `MaterialTheme.colorScheme` instead of hard-coded colours
  - ✓ Apply `MaterialTheme.typography` consistently
  - ✓ Use defined shapes for cards and buttons
- 

### Accessibility Checks

- ✓ Provide **motion reduction option** if animations are disabled
  - ✓ Ensure **text readability in both themes**
- 

### Performance Checks

- ✓ Use `LazyColumn` for efficient lists
  - ✓ Avoid complex animations in large datasets
  - ✓ Use optimized image sizes for faster loading
- 

### Edge Case Test Scenarios

#### Test Case 1 — Theme Change During Animation

Scenario:

User switches from light theme to dark theme while card expansion animation is running.

Expected Result:

UI should smoothly switch theme without breaking animation.

---

#### Test Case 2 — Rapid Favourite Toggle

Scenario:

User taps favourite icon repeatedly on a slow device.

Expected Result:

App should maintain correct favourite state and animation should not freeze.

---

## 6. Reflection

Integrating theming and animation together was challenging because both affect visual behaviour and state changes in the UI. Ensuring that animations respond correctly while maintaining consistent theme styling required careful planning of state variables and theme tokens.

When combined properly, theming provides **visual consistency**, while animations improve **interaction clarity and responsiveness**, making the UI feel more engaging and polished. However, developers must avoid overusing animations, as excessive motion can harm performance and distract users. Future developers should prioritize meaningful animations that enhance usability rather than adding motion purely for decoration.